CONVOYGUARD

STATUS: 100% CORE COMPLETE

Mobile Application Development - VII Semester Final Project Report

Framework: Flutter 3.x (Dart 3.x) **Backend:** Firebase (Auth, Firestore, FCM) **Target Platform:** Android / iOS / Web

Generated Date: October 5, 2026

📌 Executive Summary

ConvoyGuard is an end-to-end mobile safety and live coordination application engineered for group motorcycle riding. It solves critical challenges faced by riders in convoy groups—including rider separation, unexpected vehicle breakdowns, emergency crashes, and lack of real-time spatial awareness—by combining real-time GPS telemetry, custom Firestore security rules, automated stillness detection, push notification broadcasting, and emergency assistance mechanisms.

All core architecture layers, user interfaces, backend services, safety rules, and local notification dispatchers have been fully implemented and verified in the codebase.

📊 Module Completion Status

Module / Subsystem	Key Components & Services	Status	Completion
1. Auth & Rider Profiles	`AuthService`, `UserService`, Login, Register, Profile Screen	**Completed**	100%
2. Convoy & Ride Management	`RideService`, Create Ride, Join by 6-Char Code, Breakpoints, Summary	**Completed**	100%
3. Real-Time GPS & Mapping	`LocationService`, `LiveLocationService`, Google Maps, Speed Badges	**Completed**	100%
4. Safety System & SOS	`SafetyService`, `NotificationService`, Stillness Check, Emergency Screen	**Completed**	100%
5. Navigation & UI System	`MainNavigationScreen`, `DashboardScreen`, Custom Dark Theme	**Completed**	100%
6. Backend Security & Rules	`firestore.rules`, FCM Push Integration, Local Notifications	**Completed**	100%

🛠️ Detailed Module Breakdown & Features

🔐 1. Auth & Profile System

• Firebase Email & Password authentication with user-friendly error translations.
• Profile CRUD: Name, Phone, Motorcycle Model, Emergency Contact (Name, Relationship, Phone).
• Automatic FCM push token registration in Firestore user profile.

`lib/services/auth_service.dart`
`lib/services/user_service.dart`

🏁 2. Convoy & Ride Operations

• Ride creation with title, start location, destination, and dynamic break points.
• Auto-generation of unique 6-character uppercase Ride Code (e.g. X7K9P2).
• Search & join convoy by ride code.
• Status transition: Scheduled ➔ Active ➔ Completed (with analytics summary).

`lib/services/ride_service.dart`
`lib/screens/ride/`

📍 3. Live GPS Telemetry & Maps

• GPS permission handling with direct prompts to open device settings if denied.
• Real-time speed calculation (km/h) and position streaming.
• Live rider markers with custom speed badges on Google Maps.
• Auto-stop tracking option preserving last known coordinates.

`lib/services/location_service.dart`
`lib/screens/ride/live_map_screen.dart`

🚨 4. Rider Safety & Emergency SOS

• Safety alerts subcollection streaming: Stillness, No Response, Manual SOS, Emergency.
• Stillness & speed monitoring: automatically prompts rider countdown before escalating.
• Local high-priority heads-up notifications with custom sound & vibration.
• One-tap emergency calling to 911 / Emergency Contact via `url_launcher`.

`lib/services/safety_service.dart`
`lib/services/notification_service.dart`

🔒 Firestore Security Architecture

The backend safety and data isolation rules are defined in `firestore.rules`:

Collection Path	Read Access	Write Access
`users/{userId}`	Authenticated Users	Owner only (`auth.uid == userId`)
`rides/{rideId}`	Authenticated Users	Admin for setup/status; Non-admins can self-join via UID addition
`rides/{rideId}/live_locations/{userUID}`	Convoy Members only	Rider only (`auth.uid == userUID` & active member)
`rides/{rideId}/safety_alerts/{alertId}`	Convoy Members only	Rider creates alert for self; Convoy members acknowledge/resolve

📁 Source Code File Index

File Path	Type	Description
`lib/main.dart`	Entry Point	Firebase init, route generator, dark theme initialization
`lib/models/ride_model.dart`	Data Model	Ride details, ride code, status lifecycle, breakpoints, summary metrics
`lib/models/safety_alert_model.dart`	Data Model	Alert document structure (stillness, manual SOS, acknowledge/resolve status)
`lib/models/live_location_model.dart`	Data Model	Live coordinates, speed (km/h), accuracy, tracking state
`lib/services/auth_service.dart`	Service	Firebase authentication login/register/logout & error translating
`lib/services/ride_service.dart`	Service	Create, join by code, start/end ride, breakpoint management
`lib/services/location_service.dart`	Service	GPS permissions, location availability check, high accuracy position stream
`lib/services/live_location_service.dart`	Service	Sync live telemetry to Firestore subcollection
`lib/services/safety_service.dart`	Service	Safety alert creation, status stream, acknowledge/resolve operations
`lib/services/notification_service.dart`	Service	FCM push & FlutterLocalNotifications heads-up alert dispatch
`lib/screens/dashboard/dashboard_screen.dart`	UI Screen	Active ride card, SOS trigger button, ride history list
`lib/screens/ride/ride_group_screen.dart`	UI Screen	Convoy room, member status, ride code sharing, alert feed, admin controls
`lib/screens/ride/live_map_screen.dart`	UI Screen	Google Maps tracking, live rider markers, speed badges, polyline route
`lib/screens/safety/emergency_assistance_screen.dart`	UI Screen	Emergency dashboard, one-tap calling, SOS alert manager

🚀 Execution & Verification Guide

To run the application locally on an Android emulator, iOS simulator, or connected device:

```
# 1. Fetch dependencies
flutter pub get

# 2. Run the application
flutter run
```

Note on Firebase: Place your `google-services.json` (Android) or `GoogleService-Info.plist` (iOS) in the project configuration directories for live authentication & push notification testing.
